

SIR SYED UNIVERSITY OF ENGINEERING & TECHNOLOGY
COMPUTER SCIENCE DEPARTMENT

SPRING 2025

Design and Analysis of Algorithms.
Assignment 3

Semester: 6th

Name: Muhammad Faisal

Roll #: 2022F-BCS-152

Batch: 2022F

Section: A

Question)

Develop a menu-driven program in Python, C++, or C# that allows the user to choose from the following algorithms:

Select an option from the menu:

- 1) Linear Search**
- 2) Binary Search**
- 3) Bubble Sort**
- 4) Selection Sort**
- 5) Insertion Sort**
- 6) Merge Sort**
- 7) Naive Bayes Classifier**
- 8) Rabin-Karp String Matching**
- 9) Exit**

```
from sklearn.naive_bayes import GaussianNB
```

1. Linear Search

```
def linear_search(arr, target):  
    for i in range(len(arr)):  
        if arr[i] == target:  
            return i  
    return -1
```

2. Binary Search

```
def binary_search(arr, target):  
    arr.sort()  
    low, high = 0, len(arr) - 1  
    while low <= high:  
        mid = (low + high) // 2  
        if arr[mid] == target:  
            return mid  
        elif arr[mid] < target:  
            low = mid + 1  
        else:  
            high = mid - 1  
    return -1
```

3. Bubble Sort

```
def bubble_sort(arr):  
    n = len(arr)  
    for i in range(n):  
        for j in range(n - i - 1):  
            if arr[j] > arr[j + 1]:  
                arr[j], arr[j + 1] = arr[j + 1], arr[j]  
    return arr
```

4. Selection Sort

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n):
        min_idx = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr
```

5. Insertion Sort

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
    return arr
```

6. Merge Sort

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    L = merge_sort(arr[:mid])
    R = merge_sort(arr[mid:])
    return merge(L, R)
```

```
def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
```

7. Naive Bayes (with user input)

```
def naive_bayes_from_user():
    print("\n--- Naive Bayes Classifier (User Input) ---")

    n = int(input("Enter number of training samples: "))
    m = int(input("Enter number of features per sample: "))

    X = []
    y = []

    print("Enter training data (features followed by label):")
    for i in range(n):
```

```

row = list(map(float, input(f"Sample {i+1}: ").split()))
if len(row) != m + 1:
    print("Invalid input. Each row must have features + 1 label.")
    return
X.append(row[:m])
y.append(int(row[m]))

model = GaussianNB()
model.fit(X, y)

t = int(input("\nEnter number of test samples: "))
test_data = []

for i in range(t):
    row = list(map(float, input(f"Test Sample {i+1} (only features): ").split()))
    if len(row) != m:
        print("Invalid input. Each test sample must have", m, "features.")
        return
    test_data.append(row)

predictions = model.predict(test_data)
print("\nPredicted labels:", predictions)

```

8. Rabin-Karp Algorithm

```

def rabin_karp(text, pattern, prime=101):
    n = len(text)
    m = len(pattern)
    d = 256
    h = pow(d, m - 1) % prime
    p = 0
    t = 0

    for i in range(m):
        p = (d * p + ord(pattern[i])) % prime
        t = (d * t + ord(text[i])) % prime

    for i in range(n - m + 1):
        if p == t:
            if text[i:i + m] == pattern:
                return i
        if i < n - m:
            t = (d * (t - ord(text[i]) * h) + ord(text[i + m])) % prime
            if t < 0:
                t += prime
    return -1

```

Menu-driven program

```

while True:
    print("\nSelect a choice from menu")
    print("1) Linear Search")
    print("2) Binary Search")
    print("3) Bubble Sort")
    print("4) Selection Sort")
    print("5) Insertion Sort")
    print("6) Merge Sort")
    print("7) Naive Bayes")
    print("8) Rabin Karp")

```

```

print("9) Exit")

choice = input("Enter your choice (1-9): ")

if choice == '1':
    arr = list(map(int, input("Enter elements: ").split()))
    target = int(input("Enter element to search: "))
    result = linear_search(arr, target)
    print("Found at index:" if result != -1 else "Not found.", result)

elif choice == '2':
    arr = list(map(int, input("Enter sorted elements: ").split()))
    target = int(input("Enter element to search: "))
    result = binary_search(arr, target)
    print("Found at index:" if result != -1 else "Not found.", result)

elif choice == '3':
    arr = list(map(int, input("Enter elements: ").split()))
    print("Sorted:", bubble_sort(arr))

elif choice == '4':
    arr = list(map(int, input("Enter elements: ").split()))
    print("Sorted:", selection_sort(arr))

elif choice == '5':
    arr = list(map(int, input("Enter elements: ").split()))
    print("Sorted:", insertion_sort(arr))

elif choice == '6':
    arr = list(map(int, input("Enter elements: ").split()))
    print("Sorted:", merge_sort(arr))

elif choice == '7':
    naive_bayes_from_user()

elif choice == '8':
    text = input("Enter text: ")
    pattern = input("Enter pattern: ")
    result = rabin_karp(text, pattern)
    print("Pattern found at index:" if result != -1 else "Pattern not found.", result)

elif choice == '9':
    print("Exiting...")
    break

else:
    print("Invalid choice. Please enter between 1 and 9.")

```

Select a choice from menu

- 1) Linear Search
- 2) Binary Search
- 3) Bubble Sort
- 4) Selection Sort
- 5) Insertion Sort
- 6) Merge Sort
- 7) Naive Bayes
- 8) Rabin Karp
- 9) Exit

Enter your choice (1-9): █

```
Enter your choice (1-9): 1
Enter elements: 2 3 4 5 6
Enter element to search: 4
Found at index: 2
```

```
Enter your choice (1-9): 2
Enter sorted elements: 2 4 6 7 8
Enter element to search: 4
Found at index: 1
```

```
Enter your choice (1-9): 3
Enter elements: 2 4 6 8 9 1 5
Sorted: [1, 2, 4, 5, 6, 8, 9]
```

```
Enter your choice (1-9): 4
Enter elements: 1 5 7 8 1 9 6 0 5
Sorted: [0, 1, 1, 5, 5, 6, 7, 8, 9]
```

```
Enter your choice (1-9): 5
Enter elements: 2 7 9 5 4 1 0 7
Sorted: [0, 1, 2, 4, 5, 7, 7, 9]
```

```
Enter your choice (1-9): 6
Enter elements: 2 3 8 7 5 1 0
Sorted: [0, 1, 2, 3, 5, 7, 8]
```

```
Enter your choice (1-9): 7

--- Naive Bayes Classifier (User Input) ---
Enter number of training samples: 3
Enter number of features per sample: 2
Enter training data (features followed by label):
Sample 1: 5.1 3.5 0
Sample 2: 4.9 3.0 0
Sample 3: 6.2 3.4 1
```

```
Enter number of test samples: 2
Test Sample 1 (only features): 5.0 3.2
Test Sample 2 (only features): 6.1 3.3

Predicted labels: [0 0]
```

```
Enter your choice (1-9): 8
Enter text: helloworld
Enter pattern: wo
Pattern found at index: 5
```

```
Enter your choice (1-9): 9
Exiting...
```